

Exam: Time Series Visualization with Bokeh

This exam tests your ability to visualize time series data using the Bokeh library. You will be working with the "Daily Minimum Temperatures in Melbourne" dataset. For each question, provide the Python code using Bokeh to generate the requested visualization.

Dataset: daily-minimum-temperatures-in-melbourne.csv

```
In [12]: # Importations nécessaires
import pandas as pd
from bokeh.plotting import figure, show
from bokeh.io import output_notebook
from bokeh.models import ColumnDataSource, HoverTool, DatetimeTickFormatter
from bokeh.layouts import row, column
from bokeh.transform import factor_cmap

# Configuration de l'affichage dans le notebook
output_notebook()

# Étape 1 : Chargement des données
donnees = pd.read_csv("../datasets/daily-minimum-temperatures-in-melbourne.csv")

# Étape 2 : Nettoyage et formatage
donnees.columns = ['Date', 'Temperature']
donnees['Date'] = pd.to_datetime(donnees['Date'])

# Retrait des caractères '?' s'il y en a et conversion en numérique
donnees['Temperature'] = donnees['Temperature'].astype(str).str.replace('?', '', regex=False)
donnees['Temperature'] = pd.to_numeric(donnees['Temperature'])

# Affichage des premières lignes pour vérifier
donnees.head()
```



BokehJS 3.9.0 successfully loaded.

```
Out [12]:
```

	Date	Temperature
0	1981-01-01	20.7
1	1981-01-02	17.9
2	1981-01-03	18.8
3	1981-01-04	14.6
4	1981-01-05	15.8

Question 1: Basic Time Series Line Plot

Create a basic line plot showing the daily minimum temperature over time.

- Use the 'Date' column on the x-axis and the 'Temperature' column on the y-axis.
- Set the plot title to "Daily Minimum Temperatures".
- Label the x-axis as "Date" and the y-axis as "Temperature (°C)".
- Add tooltips to display the date and temperature when hovering over the line.
- Enable pan, wheel zoom, and reset tools.

```
In [13]: # Paramétrage de la source de données pour Bokeh
source_ligne = ColumnDataSource(donnees)

# Création du graphique (plot)
graph_1 = figure(
    title="Températures Minimales Journalières à Melbourne",
    x_axis_type="datetime",
    tools="pan,wheel_zoom,reset",
    width=800, height=400
)

# Traçage de la ligne
ligne = graph_1.line(
    x="Date",
    y="Temperature",
    source=source_ligne,
    line_width=1.5,
```

```

    color="navy"
)

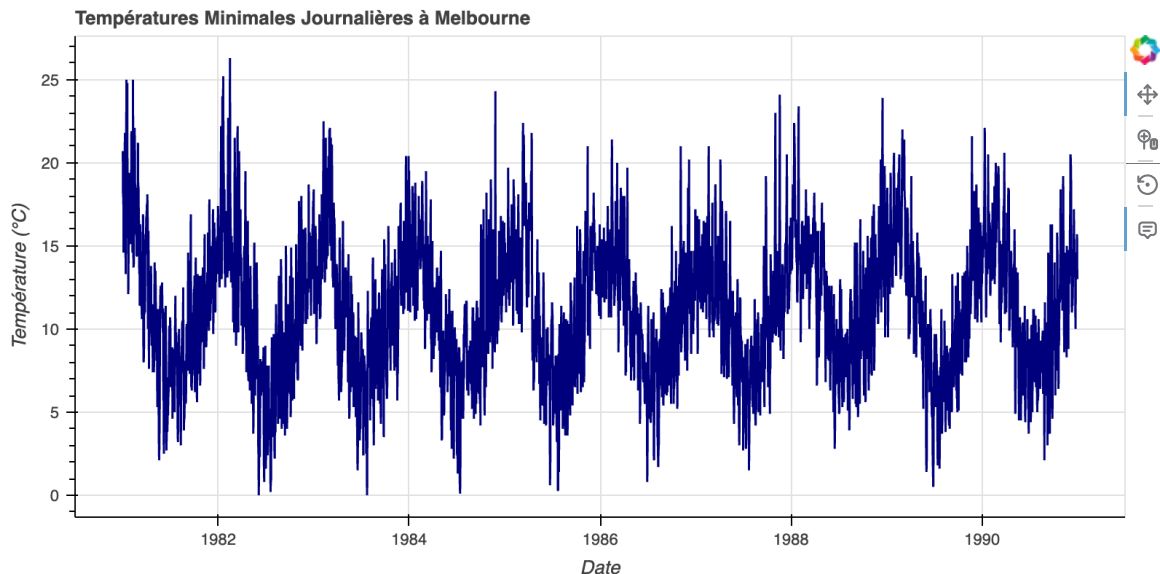
# Configuration de l'infobulle (hover)
infobulle_1 = HoverTool(
    renderers=[ligne],
    tooltips=[("Date", "@Date{%F}"), ("Temp.", "@Temperature{0.0} °C")],
    formatters={"@Date": "datetime"},
    mode="vline" # L'infobulle s'affiche verticalement par rapport à la souris
)

graph_1.add_tools(infobulle_1)

# Personnalisation des axes
graph_1.xaxis.axis_label = "Date"
graph_1.yaxis.axis_label = "Température (°C)"

show(graph_1)

```



Question 2: Rolling Average

Calculate the 30-day rolling average of the daily minimum temperature and plot it alongside the original temperature data.

- Create a new column 'Rolling_Avg' in the DataFrame containing the 30-day rolling average.
- Plot both the original 'Temperature' and the 'Rolling_Avg' on the same plot.
- Use different colors and line styles to distinguish between the two.
- Add a legend to the plot to label the lines.
- Add tooltips to display the date, original temperature, and rolling average.

```

In [14]: # Calcul de la moyenne glissante sur 30 jours
donnees["Moyenne_Glissante"] = donnees["Temperature"].rolling(window=30, min_periods=1).mean()

# Mise à jour de la source pour prendre en compte la nouvelle colonne
source_mm = ColumnDataSource(donnees)

graph_2 = figure(
    title="Température Journalière et Moyenne Mobile sur 30 jours",
    x_axis_type="datetime",
    tools="pan,wheel_zoom,reset",
    width=900,
    height=400
)

# Courbe de température brute (bleue, plus fine)
ligne_brute = graph_2.line(
    x="Date",
    y="Temperature",
    source=source_mm,
    line_width=1.5,
    color="steelblue",
    legend_label="Température (réelle)",
    alpha=0.6 # Légère transparence
)

# Courbe lisse (moyenne mobile) (rouge et en pointillés)

```

```

ligne_lisse = graph_2.line(
    x="Date",
    y="Moyenne_Glissante",
    source=source_mm,
    line_width=2.5,
    color="crimson",
    line_dash="dashed",
    legend_label="Moyenne Glissante (30J)"
)

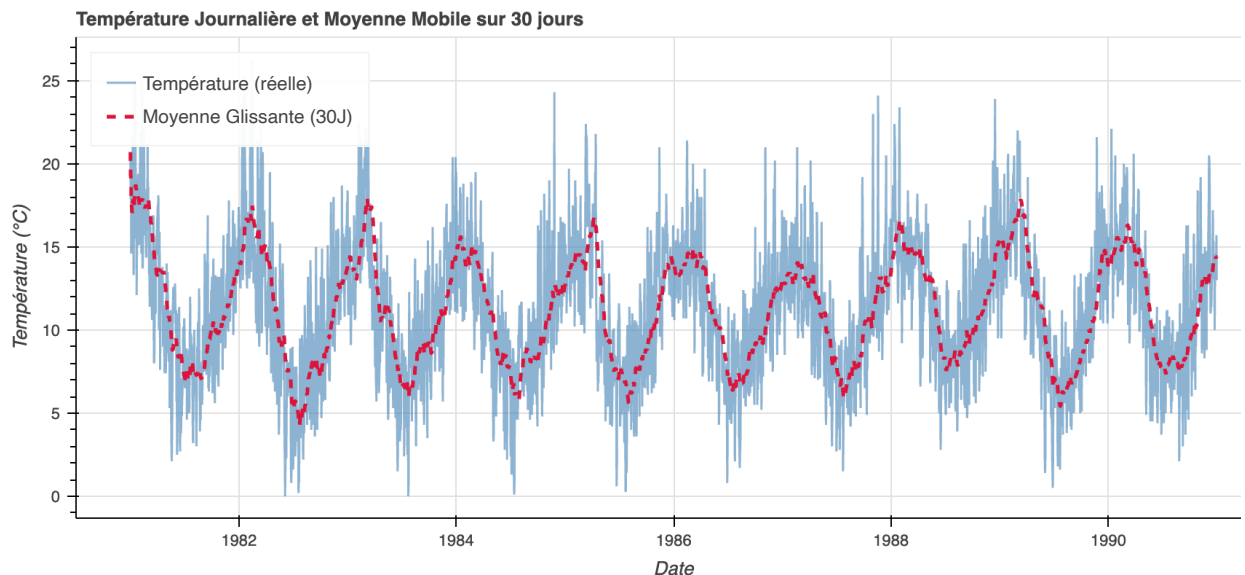
# Configuration dynamique des infobulles sur le survol
infobulle_2 = HoverTool(
    renderers=[ligne_brute, ligne_lisse],
    tooltips=[
        ("Jour", "@Date{%F}"),
        ("Init.", "@Temperature{0.0} °C"),
        ("Moy.", "@Moyenne_Glissante{0.0} °C")
    ],
    formatters={"@Date": "datetime"},
    mode="vline"
)

graph_2.add_tools(infobulle_2)

# Positionnement et style de la légende et axes
graph_2.legend.location = "top_left"
graph_2.xaxis.axis_label = "Date"
graph_2.yaxis.axis_label = "Température (°C)"

show(graph_2)

```



Question 3: Monthly Box Plots

Create box plots to visualize the distribution of temperatures for each month.

- Extract the month from the 'Date' column and create a new 'Month' column.
- Group the data by 'Month' and prepare it for plotting.
- Use Bokeh's box plot elements to visualize the distribution.
- Label the x-axis with month names and the y-axis with "Temperature (°C)".
- Add tooltips to display the month and relevant statistical values (min, max, median).

```

In [15]: # Question 3: Boîtes à moustaches (Boxplots) par mois
donnees["Mois"] = donnees["Date"].dt.month_name().str.slice(0, 3)
ordre_mois = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]

# Calcul des statistiques mois par mois
groupe_mois = donnees.groupby("Mois")["Temperature"]
stats_mensuelles = groupe_mois.describe().reset_index()

# Extraction des quartiles pour la construction de la boîte
stats_mensuelles["q1"] = groupe_mois.quantile(0.25).values
stats_mensuelles["q2"] = groupe_mois.quantile(0.50).values # Médiane
stats_mensuelles["q3"] = groupe_mois.quantile(0.75).values

```

```

# Limites des valeurs (moustache supérieure et inférieure)
stats_mensuelles["haut"] = (stats_mensuelles["q3"] + 1.5 * (stats_mensuelles["q3"] - stats_mensuelles["q1"]))
stats_mensuelles["bas"] = (stats_mensuelles["q1"] - 1.5 * (stats_mensuelles["q3"] - stats_mensuelles["q1"]))

# Réorganisation de manière chronologique
stats_mensuelles["Mois"] = pd.Categorical(stats_mensuelles["Mois"], categories=ordre_mois, ordered=True)
stats_mensuelles = stats_mensuelles.sort_values("Mois")
stats_mensuelles["Mois"] = stats_mensuelles["Mois"].astype(str) # Converti en texte pour Bokeh

source_box = ColumnDataSource(stats_mensuelles)

# Initialisation du graphique de boîte à moustache
graph_3 = figure(
    title="Distribution mensuelle des températures",
    x_range=ordre_mois,
    tools="pan,wheel_zoom,reset",
    width=850,
    height=450
)

# Configuration de l'affichage des éléments des boxplots (Segment + vbar + rect)
graph_3.segment(x0="Mois", y0="haut", x1="Mois", y1="q3", source=source_box, line_color="black") # Ligne supérieure
graph_3.segment(x0="Mois", y0="bas", x1="Mois", y1="q1", source=source_box, line_color="black") # Ligne inférieure
graph_3.vbar(x="Mois", width=0.6, top="q3", bottom="q2", source=source_box, fill_color="#E08E79") # Haut boxplot
graph_3.vbar(x="Mois", width=0.6, top="q2", bottom="q1", source=source_box, fill_color="#3B8686") # Bas boxplot

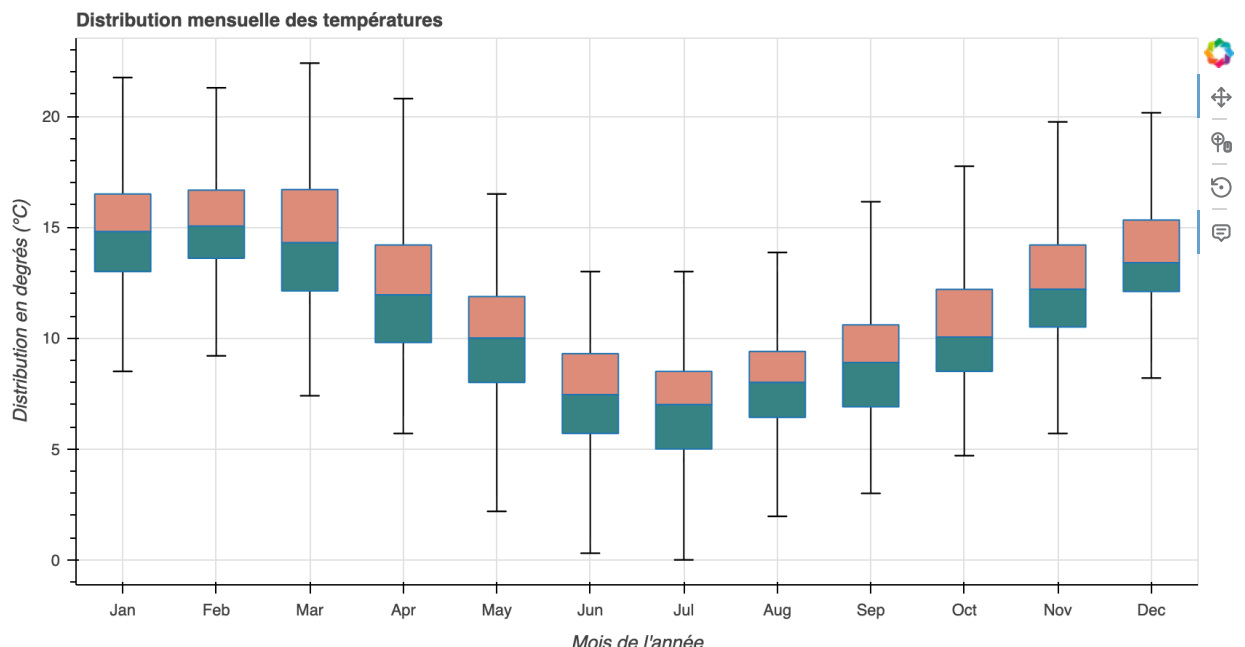
# Barres d'extrémité sur les moustaches
graph_3.rect(x="Mois", y="bas", width=0.2, height=0.01, source=source_box, line_color="black")
graph_3.rect(x="Mois", y="haut", width=0.2, height=0.01, source=source_box, line_color="black")

# Configuration de l'infobulle pour plus de lisibilité
infobulle_3 = HoverTool(
    tooltips=[
        ("Période", "@Mois"),
        ("Minimum", "@min{0.0} °C"),
        ("Maximum", "@max{0.0} °C"),
        ("Médiane", "@q2{0.0} °C")
    ]
)

graph_3.add_tools(infobulle_3)
graph_3.xaxis.axis_label = "Mois de l'année"
graph_3.yaxis.axis_label = "Distribution en degrés (°C)"

show(graph_3)

```



Question 4: Yearly Box Plots

Create box plots to visualize the distribution of temperatures for each year, and use color mapping to highlight temperature variations.

- Extract the year from the 'Date' column and create a new 'Year' column.

- Group the data by 'Year' and prepare it for plotting.
- Use Bokeh's box plot elements to visualize the distribution for each year.
- Label the x-axis with the 'Year' and the y-axis with "Temperature (°C)".
- Use factor_cmap to color the boxes based on the median temperature of each year.
- Add tooltips to display the year and relevant statistical values (min, max, median, etc.).
- Enable pan, wheel zoom, and reset tools.

```
In [16]: # Extraction de l'année pour créer la colonne 'Année'
donnees["Année"] = donnees["Date"].dt.year.astype(str)

# Calcul des statistiques annuelles
groupe_annee = donnees.groupby("Année")["Temperature"]
stats_annuelles = groupe_annee.describe().reset_index()

# Extraction des quartiles pour la construction de la boîte
stats_annuelles["q1"] = groupe_annee.quantile(0.25).values
stats_annuelles["q2"] = groupe_annee.quantile(0.50).values # Médiane
stats_annuelles["q3"] = groupe_annee.quantile(0.75).values

# Limites des valeurs (moustache supérieure et inférieure)
stats_annuelles["haut_annee"] = (stats_annuelles["q3"] + 1.5 * (stats_annuelles["q3"] - stats_annuelles["q2"]))
stats_annuelles["bas_annee"] = (stats_annuelles["q1"] - 1.5 * (stats_annuelles["q3"] - stats_annuelles["q1"]))

# Catégorisation des températures pour jouer avec les couleurs (bas, moyen, haut)
seuil_bas = stats_annuelles["q2"].quantile(0.33)
seuil_haut = stats_annuelles["q2"].quantile(0.66)
stats_annuelles["niveau_mediane"] = pd.cut(
    stats_annuelles["q2"],
    bins=[-1e9, seuil_bas, seuil_haut, 1e9],
    labels=["Bas", "Moyen", "Élevé"],
    include_lowest=True
).astype(str)

ordre_annee = stats_annuelles["Année"].tolist()

# Source des données pour cette visualisation
source_annee = ColumnDataSource(stats_annuelles)

# Définition de la palette de couleurs en fonction des niveaux des températures
carte_couleur = factor_cmap("niveau_mediane", palette=["#2171b5", "#fec44f", "#cb181d"], factors=["Bas", "Moyen", "Élevé"])

# Initialisation du graphique de boîte à moustache annuel
graph_4 = figure(
    title="Analyse de la température Année après Année",
    x_range=ordre_annee,
    tools="pan,wheel_zoom,reset",
    width=900,
    height=450
)

# Configuration de l'affichage des éléments des boxplots annuels
graph_4.segment(x0="Année", y0="haut_annee", x1="Année", y1="q3", source=source_annee, line_color="black")
graph_4.segment(x0="Année", y0="bas_annee", x1="Année", y1="q1", source=source_annee, line_color="black")

# Couleur dynamique pour le corps de la boîte
boite = graph_4.vbar(
    x="Année",
    width=0.6,
    top="q3",
    bottom="q1",
    source=source_annee,
    fill_color=carte_couleur
)

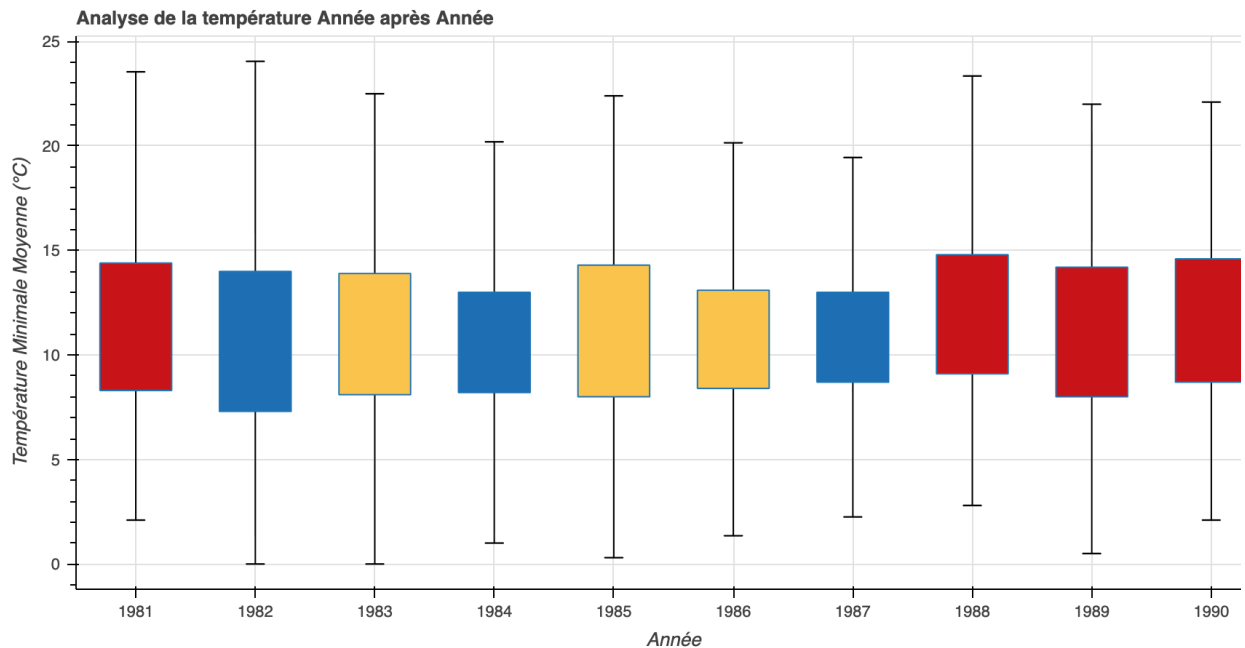
# Barres d'extrémité sur les moustaches
graph_4.rect(x="Année", y="bas_annee", width=0.15, height=0.01, source=source_annee, line_color="black")
graph_4.rect(x="Année", y="haut_annee", width=0.15, height=0.01, source=source_annee, line_color="black")

# Configuration de l'infobulle pour plus de détails
infobulle_4 = HoverTool(
    renderers=[boite],
    tooltips=[
        ("Année", "@Année"),
        ("Minimum", "@min{0.0} °C"),
        ("Maximum", "@max{0.0} °C"),
        ("Médiane", "@q2{0.0} °C"),
        ("1er quartile", "@q1{0.0} °C"),
        ("3ème quartile", "@q3{0.0} °C"),
        ("Catégorie", "@niveau_mediane")
    ]
)
```

```
)

graph_4.add_tools(Infobulle_4)
graph_4.xaxis.axis_label = "Année"
graph_4.yaxis.axis_label = "Température Minimale Moyenne (°C)"

show(graph_4)
```



Question 5: Interactive Time Range Selection

Create an interactive line plot where the user can select a specific time range to view using a date range slider.

- Create a basic line plot of 'Temperature' over 'Date'.
- Implement a date range slider using Bokeh widgets to allow users to select a start and end date.
- Update the plot dynamically based on the selected date range.
- Add tooltips to display the date and temperature.
- Enable pan, wheel zoom, and reset tools.

```
In [17]: # Importation du widget de plage de dates
from bokeh.models import DateRangeSlider

# Préparation de la source pour la courbe interactive
source_interactive = ColumnDataSource(donnees)

graph_5 = figure(
    title="Analyse Temporelle Interactive (Températures Min)",
    x_axis_type="datetime",
    tools="pan,wheel_zoom,reset",
    width=800,
    height=400
)

# Tracé principal
graph_5.line(
    x="Date",
    y="Temperature",
    source=source_interactive,
    line_width=1.5,
    color="darkgreen"
)

# Infobulles pour chaque point
infobulle_5 = HoverTool(
    tooltips=[("Jour exact", "@Date{%F}"), ("Météo", "@Temperature{0.0} °C")],
    formatters={"@Date": "datetime"},
    mode="vline"
)

graph_5.add_tools(infobulle_5)
graph_5.xaxis.axis_label = "Temps"
graph_5.yaxis.axis_label = "Température Minimale (°C)"
```

```
# Création du curseur interactif permettant de sélectionner les dates (Date Range Slider)
curseur_dates = DateRangeSlider(
    title="Sélectionner une plage de temps",
    start=donnees["Date"].min(),
    end=donnees["Date"].max(),
    value=(donnees["Date"].min(), donnees["Date"].max()),
    width=400
)

# Relier la plage de sélection des dates (js_link) au graphique
curseur_dates.js_link("value", graph_5.x_range, "start", attr_selector=0)
curseur_dates.js_link("value", graph_5.x_range, "end", attr_selector=1)

# Affichage simultané du curseur et du graphique avec "column"
show(column(curseur_dates, graph_5))
```

Question 6: Time Series Decomposition Visualization

Perform a simple time series decomposition to visualize the trend and seasonality components of the temperature data.

- Resample the data to monthly frequency and calculate the monthly average temperature.
- Use a simple moving average to estimate the trend component.
- Calculate the seasonal component by subtracting the trend from the original monthly data.
- Create three separate Bokeh plots: one for the original monthly data, one for the trend, and one for the seasonal component.
- Ensure the plots are aligned and share the same x-axis (Date).
- Add tooltips to each plot to display the date and corresponding value.
- Enable pan, wheel zoom, and reset tools for each plot.

```
In [18]: # Réduction de la granularité (passage en données mensuelles et calcul de la moyenne par mois)
donnees_mensuelles = donnees.set_index("Date")["Temperature"].resample("ME").mean().reset_index()

# Tendance : Estimation à l'aide d'une moyenne trimestrielle glissante (sur 12 mois)
donnees_mensuelles["Tendance"] = donnees_mensuelles["Temperature"].rolling(window=12, min_periods=1, center=0).mean()

# Composante saisonnière perçue au travers de la différence: Température Réelle - Tendance
donnees_mensuelles["Saisonnalité"] = donnees_mensuelles["Temperature"] - donnees_mensuelles["Tendance"]

source_decomposition = ColumnDataSource(donnees_mensuelles)

# Graphique 1 : Moyenne mensuelle originale
graph_6a = figure(
    title="Température moyenne globale mensuelle",
    x_axis_type="datetime",
    tools="pan,wheel_zoom,reset",
    width=800, height=250
)
graph_6a.line("Date", "Temperature", source=source_decomposition, line_width=2, color="black")
infobulle_6a = HoverTool(
    tooltips=[("Date", "@Date{%F}"), ("Valeur mensuelle", "@Temperature{0.0} °C")],
    formatters={"@Date": "datetime"},
    mode="vline"
)
graph_6a.add_tools(infobulle_6a)
graph_6a.yaxis.axis_label = "En (°C)"

# Graphique 2 : Tendance de fond (Trend)
# Lien avec le 1er graphique sur l'axe X pour un zoom/déplacement synchronisé
graph_6b = figure(
    title="Estimation de la Tendance",
    x_axis_type="datetime",
    tools="pan,wheel_zoom,reset",
    x_range=graph_6a.x_range,
    width=800, height=250
)
graph_6b.line("Date", "Tendance", source=source_decomposition, line_width=2, color="orange")
infobulle_6b = HoverTool(
    tooltips=[("Date", "@Date{%F}"), ("Tendance long-terme", "@Tendance{0.0} °C")],
    formatters={"@Date": "datetime"},
    mode="vline"
)
graph_6b.add_tools(infobulle_6b)
graph_6b.yaxis.axis_label = "Tendance"

# Graphique 3 : Bruits et saisonnalités (Seasonality)
graph_6c = figure(
    title="Effet Saisonnier",
```

```

x_axis_type="datetime",
tools="pan,wheel_zoom,reset",
x_range=graph_6a.x_range, # Partage de l'axe temporel
width=800, height=250
)
graph_6c.line("Date", "Saisonnalite", source=source_decomposition, line_width=2, color="teal")
infobulle_6c = HoverTool(
    tooltips=[("Date", "@Date{%F}"), ("Saisonnalité mensuelle", "@Saisonnalite{0.0} °C")],
    formatters={"@Date": "datetime"},
    mode="vline"
)
graph_6c.add_tools(infobulle_6c)
graph_6c.xaxis.axis_label = "Temps"
graph_6c.yaxis.axis_label = "Saison "

# Grouper et afficher les 3 sur une même colonne
show(column(graph_6a, graph_6b, graph_6c))

```

